

By GROUP 6

Contents

4. System Modelling and Design of netDrive	2
4.1 Data Flow Diagram 4.1.1 Introduction The NetDrive system is a crowdsensed mobile network measurement platform designed to collect, process, and analyze radio frequency (RF) data from user devices. This report presents two key UML diagrams, the Data Flow Diagram (DFD) and the Deployment Diagram which together illustrate how data moves through the system and how its components are physically deployed.	2
2. Overview of the Crowdsensed Drive Test System (netDrive)	2
4.2.2. Explanation of the Data Flows	3
a. The Volunteer Contributor (The Mobile User)	3
b. Telecommunication Regulatory Boards (e.g. ART Cameroon)	3
c. Infrastructure Providers & Mobile Network Operators (MNOs)	4
4.3 Data Flow Diagram (DFD)	4
4.3.1 Purpose The Data Flow Diagram represents the logical flow of data within the NetDrive system, showing how information travels from mobile devices to backend servers and finally to analysts. It highlights the processes, data stores, and external entities involved in the data collection and analysis pipeline.	5
4.4 Use Case Diagram	5
4.4.1 Introduction	6
4.4.2 System Actors	6
4.4.3 Major Use Cases	6
4.4.4 Diagrammatic Representation of Use Case Diagram	6
4.4.5 Explanation of the Use Case Diagram	7
4.6.3 Classes, Attributes, and Methods	11
User Management User Represents any person who has installed the application. Every user is automatically a data contributor, no separate tester role exists. The consentGiven flag must be true before any data collection begins.	12
Privacy & Security Consent Manager	12
Core Data	13
Storage & Synchronization LocalDatabase Provides offline-first persistent storage using an on-device SQLite database. Every anonymized sample is written locally immediately after collection, regardless of internet availability. Supports encryption at rest and automatic retention-based cleanup (default 30 days)	14
Backend & Analysis	15

4.6.4 Class Relationships	16
4.6.5 Association	16
4.6.6 Composition	16
4.6.7 Dependency	17
4.6.8 Complete Relationship Summary	17
4.6.9 End-to-End System Flow	18
4.6.10 Design Principles Reflected in the Diagram The class structure embodies several key object-oriented design principles:.....	19
4.7 Deployment Diagram	20
4.7.1 Purpose The Deployment Diagram illustrates the physical architecture of the NetDrive system, showing how software components are distributed across hardware nodes and how they interact through secure connections.	20
4.7.2 Description	20
4.7.3 Conclusion	20
References	21

4. System Modelling and Design of netDrive

4.1 Data Flow Diagram

4.1.1 Introduction

The NetDrive system is a crowdsensed mobile network measurement platform designed to collect, process, and analyze radio frequency (RF) data from user devices. This report presents two key UML diagrams, the Data Flow Diagram (DFD) and the Deployment Diagram which together illustrate how data moves through the system and how its components are physically deployed.

2. Overview of the Crowdsensed Drive Test System (netDrive)

The center bubble represents the **Crowdsensed Drive Test Platform**. This is a decentralized software system designed to replace traditional, expensive physical drive-testing methods that require specialized vehicles and hardware.

The primary job of this system is to automatically collect real-world, localized network telemetry from mobile users acting as passive probes, process this spatial information, and deliver actionable network health intelligence to telecommunication oversight bodies and operators.

4.2 Context Diagram

4.2.1 Introduction:

A Context Diagram is a simple, high-level map used in software engineering to show the scope of a project. It answers one main question: What is inside the software system, and what is outside talking to it?

It uses only three basic elements:

- **The Center Bubble:** Represents the entire software application as one single process. It does not show internal code, databases, or hidden folders.
- **The Outside Boxes (External Entities):** Represent the people, organizations, or environments that need to interact with the app.
- **The Arrows (Data Flows):** Represent the data being passed back and forth across the system boundary.

4.2.2. Explanation of the Data Flows

The diagram maps out exactly how the system interacts with its three outside boxes:

a. The Volunteer Contributor (The Mobile User)

The volunteer contributor stands outside the system and serves as the data collection probe in the field.

- **Data In (Inputs):** The user's device automatically captures and feeds **Telemetry Data** directly into the system. This payload includes cellular performance metrics (RSRP, RSRQ, SINR, RSSI), spatiotemporal info (Latitude, Longitude, and UTC Timestamps), and device metadata (Device Model and Baseband Network Type).
- **Data Out (Outputs):** None. To maximize battery efficiency and protect user privacy, the app does not send mapping visualizers or heavy analytics back to the client handset.

b. Telecommunication Regulatory Boards (e.g. ART Cameroon)

The regulatory board acts as an independent oversight entity monitoring systemic compliance.

- **Data In (Inputs):** None. The regulatory board does not input operational data streams into the platform.

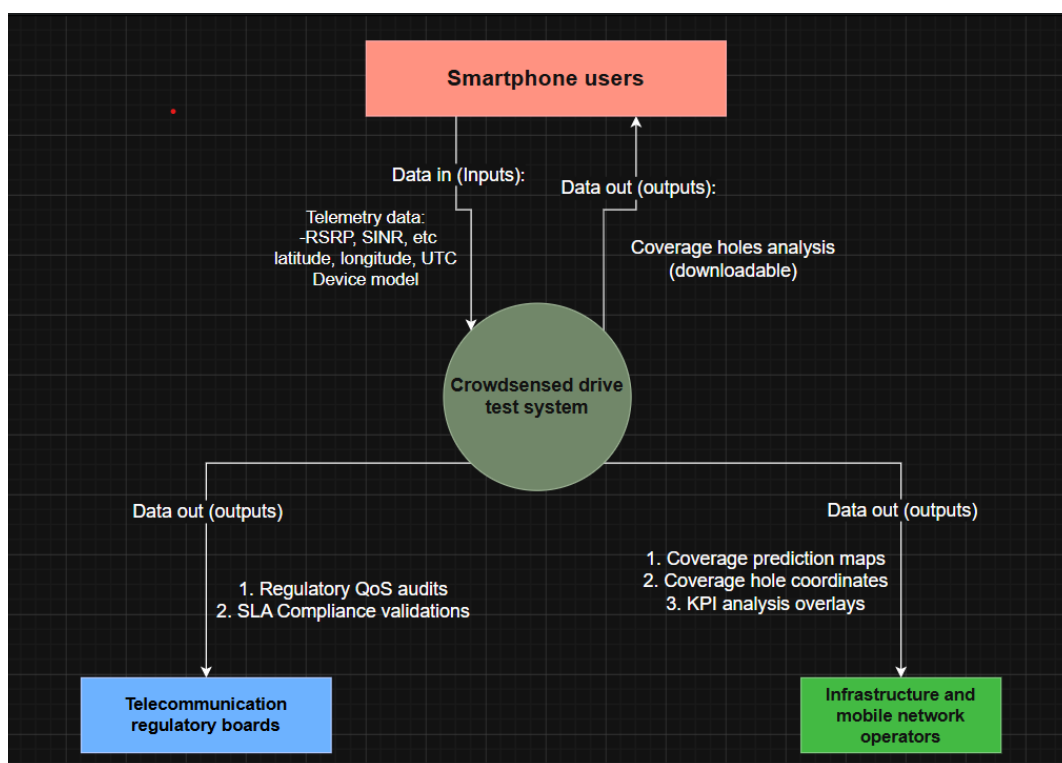
- **Data Out (Outputs): Regulatory QoS Audits.** The system delivers clean, crowdsourced Quality of Service validations and Service Level Agreement (SLA) metrics to help regulators verify carrier coverage claims without market bias.

c. Infrastructure Providers & Mobile Network Operators (MNOs)

This box represents the engineering teams at telecom networks (like MTN, Orange, Camtel) and tower companies (like IHS Towers).

- **Data In (Inputs):** None. Operators do not provide configuration profiles or direct data inputs to the platform to maintain the system's structural objectivity.
- **Data Out (Outputs): Coverage Prediction Maps.** The system pushes out high-resolution signal heatmaps and the exact coordinates of detected network **Coverage Holes**, showing operators exactly where physical tower adjustments or capital expansions are required.

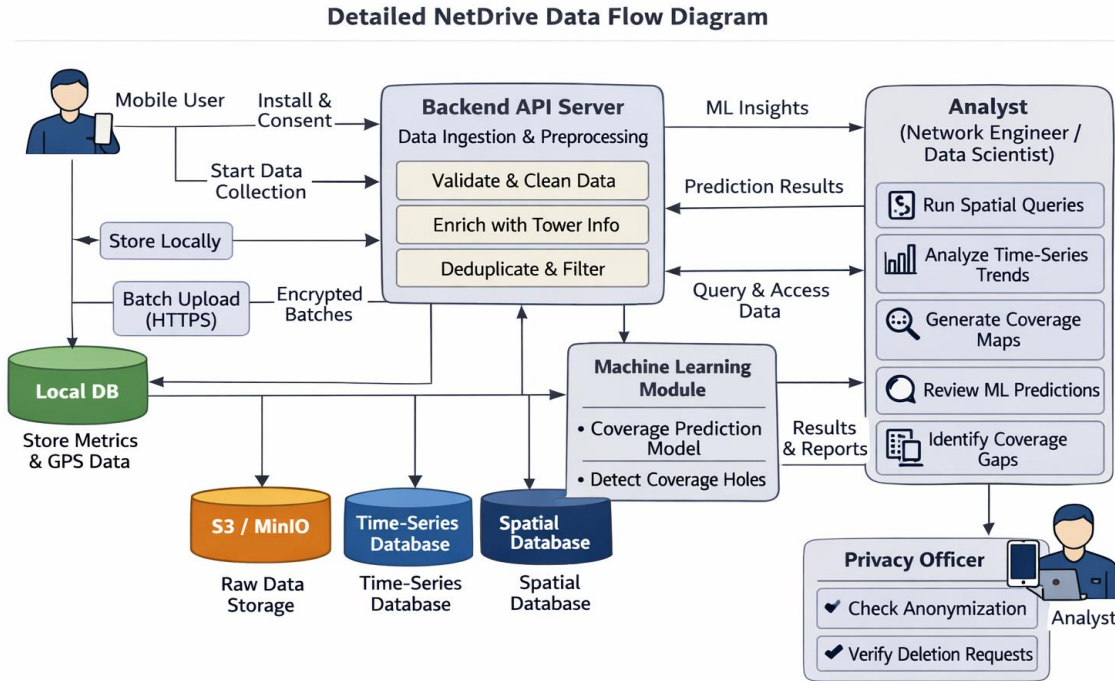
4.2.3 System Context Diagram



4.3 Data Flow Diagram (DFD)

4.3.1 Purpose

The Data Flow Diagram represents the logical flow of data within the NetDrive system, showing how information travels from mobile devices to backend servers and finally to analysts. It highlights the processes, data stores, and external entities involved in the data collection and analysis pipeline.



4.3.2 Description

- Actors:
 - Mobile User (Field Tester): Initiates data collection and provides consent.
 - Analyst (Network Engineer/Data Scientist): Reviews processed data and ML predictions.
- Processes:
 - NetDrive Mobile App: Collects RF metrics, GPS coordinates, and throughput data.
 - Backend API Server: Handles data ingestion, validation, and preprocessing.
 - Machine Learning Module: Predicts coverage holes and generates insights.
 - Analyst Platform: Visualizes coverage maps and reports.
- Data Stores:
 - Local DB: Temporary storage on the mobile device.
 - S3/MinIO: Raw data storage.
 - ClickHouse/InfluxDB: Time-series data storage.
 - PostgreSQL/PostGIS: Spatial data storage.
- Data Flows:
 - Secure data transmission via HTTPS/TLS.
 - Batch uploads from mobile devices to backend.
 - Processed data queried by analysts for visualization and reporting.

4.4 Use Case Diagram

4.4.1 Introduction

A Use Case Diagram is a UML modeling tool used to represent how different users interact with a system.

For the Crowdsensed DriveTest Mobile Application, it shows how stakeholders interact with the mobile and backend platform to collect signal measurements, monitor weak coverage zones, manage privacy compliance, and maintain the application.

This model ensures that the system requirements are clearly linked to each user before implementation begins.

4.4.2 System Actors

Field Tester – collects network measurements using the mobile app.

Operator Analyst – monitors coverage maps and analyzes network performance.

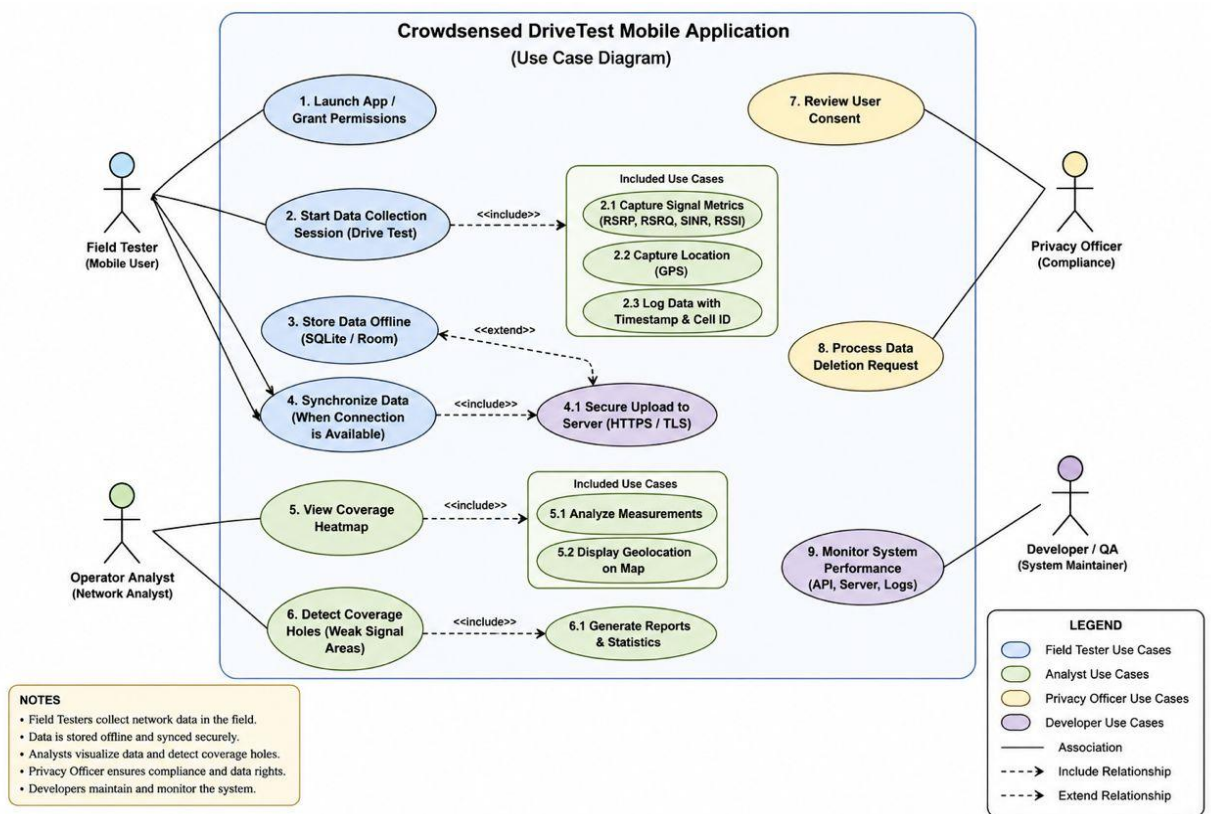
Privacy Officer – verifies consent and handles deletion requests.

Developer / QA Engineer – tests, debugs, and maintains the system.

4.4.3 Major Use Cases

- Launch/Register App
- Start Data Collection Session
- Store Data Offline
- Synchronize Data with Backend
- View Coverage Heatmap
- Detect Coverage Holes
- Review User Consent
- Process Data Deletion Request
- Monitor System Performance

4.4.4 Diagrammatic Representation of Use Case Diagram



4.4.5 Explanation of the Use Case Diagram

The use case diagram shows how different stakeholders interact with the Crowdsensed DriveTest Mobile Application. The system boundary represents the complete application, while the actors outside the boundary represent the users who interact with it. The main actors are the Field Tester, Operator Analyst, Privacy Officer, and Developer/QA.

- **Field Tester**

The Field Tester is the main mobile user. The tester launches the app, grants permissions, starts the data collection session, stores data offline when network is unavailable, and synchronizes data when connection returns. During collection, the system captures signal metrics such as RSRP, RSRQ, SINR, and RSSI, together with GPS location, timestamps, and Cell ID.

- **Operator Analyst**

The Operator Analyst uses the collected data to view coverage heatmaps and identify weak network zones. The analyst also generates reports and statistics that help evaluate network quality and support decision making.

- **Privacy Officer**

The Privacy Officer ensures that users have given consent before data is collected and

handles requests for data deletion when needed. This protects user privacy and supports compliance requirements.

- **Developer / QA**

The Developer or QA Engineer monitors system performance, including API activity, server logs, and application errors. This helps maintain reliability and quickly resolve technical issues.

4.5 Sequence Diagram

4.5.1 Introduction

A Sequence Diagram shows how users and system components interact over time. For the Crowdsensed DriveTest Mobile Application, it illustrates interactions between stakeholders, the mobile app, backend server, and monitoring services

4.5.2 System Participants

Field Tester – collects network measurements.

Mobile Application – captures and manages data.

Backend Server – stores and processes uploaded data.

Operator Analyst – analyzes network coverage.

Privacy Officer – manages consent and deletion requests.

Developer / QA – monitors system performance.

4.5.3 Sequence Diagrams

1. Data Collection & Synchronization

- Flow:
 1. Field Tester launches app.
 2. App requests permissions.
 3. Tester grants access.
 4. App captures: Signal metrics (RSRP, RSRQ, SINR, RSSI), GPS, Timestamp, Cell ID
 5. Data stored offline.
 6. App synchronizes data when internet is available.
 7. Backend validates and stores records.
- Field Tester → Mobile App: Launch App
- Mobile App → Field Tester: Request Permissions
- Field Tester → Mobile App: Grant Permissions
- Mobile App → Mobile App: Capture Measurements
- Mobile App → Local Storage: Store Offline Data
- Mobile App → Backend Server: Upload Data
- Backend Server → Mobile App: Sync Success

2. Coverage Analysis & Heatmap

Flow:

1. Analyst requests coverage data.
2. Backend retrieves measurements.
3. System analyzes metrics.
4. Heatmap and weak signal zones generated.

Operator Analyst → Backend Server: Request Coverage Data

Backend Server → Analytics: Analyze Measurements

Analytics → Backend Server: Generate Heatmap

Backend Server → Operator Analyst: Display Results

3. User Consent Management

- Flow:

1. User opens app
2. Consent request displayed.
3. User accepts/rejects.
4. Consent stored for compliance review.

- User → Mobile App: Open App

Mobile App → User: Show Consent Form

User → Mobile App: Accept/Reject

Mobile App → Backend Server: Save Consent

Privacy Officer → Backend Server: Review Consent

4. Data Deletion Request

- Flow:

1. Privacy Officer requests deletion.
2. Backend locates records.
3. Data deleted.
4. Action logged.

- Privacy Officer → Backend Server: Request Deletion

Backend Server → Database: Find/Delete Records

Backend Server → Privacy Officer: Deletion Confirmed

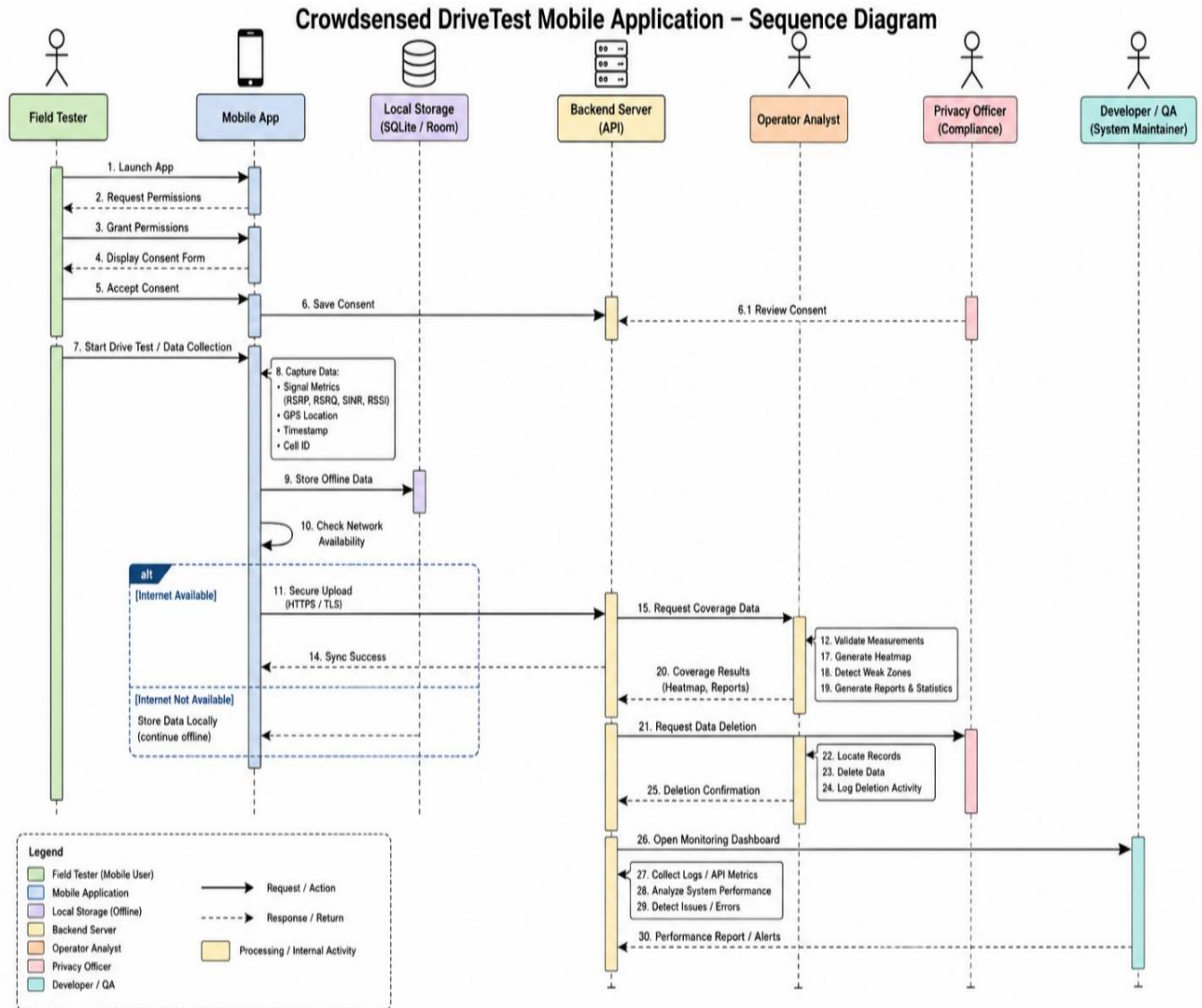
5. System Performance Monitoring

Flow:

1. Developer accesses dashboard.
2. Monitoring system collects logs and metrics.
3. Performance report generated.

Developer/QA → Monitoring Service: Open Dashboard
Monitoring Service → Backend Server: Collect Logs/API Metrics
Monitoring Service → Developer/QA: Performance Report

4.5.4 Diagrams



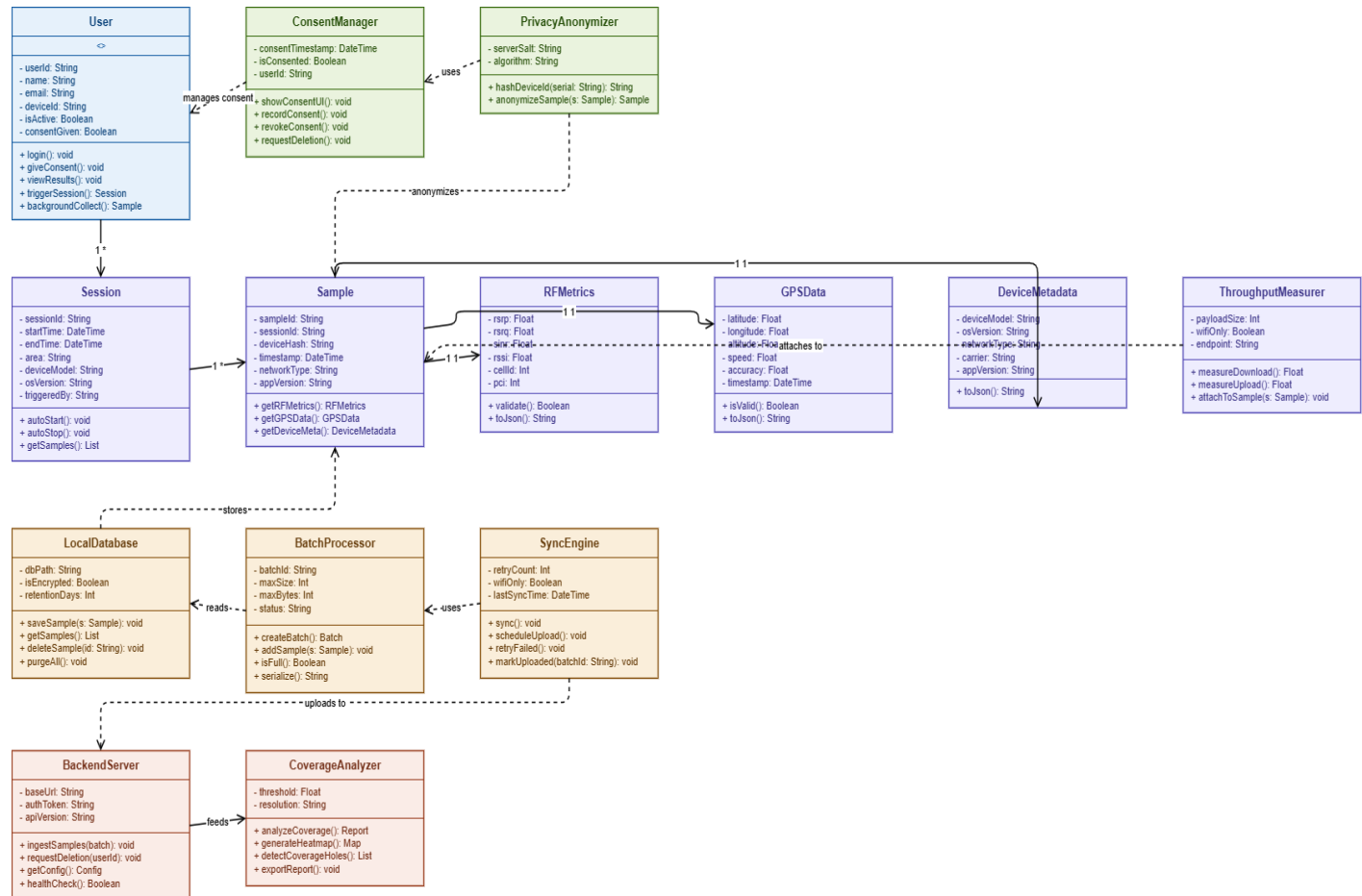
4.6 CLASS DIAGRAM

4.6.1 Introduction

A class diagram is a type of structural diagram in the Unified Modelling Language (UML). It provides a static, blueprint-level view of a software system by showing the classes that make up

the system, the data each class holds (its attributes), the actions each class can perform (its methods), and the relationships between classes.

4.6.2 Diagram



4.6.3 Classes, Attributes, and Methods

The 15 classes are organized into 5 groups based on their functional role:

Group	Classes	Responsibility
User Management	User	Represents the person using the application
Privacy & Security	ConsentManager, PrivacyAnonymizer	Manages consent and anonymises personal data

Group	Classes	Responsibility
Core Data	Session, Sample, RFMetrics, GPSData, DeviceMetadata, ThroughputMeasurer	Defines the structure of all collected data
Storage & Synchronisation	LocalDatabase, BatchProcessor, SyncEngine	Persists data locally and uploads to the backend
Backend & Analysis	BackendServer, CoverageAnalyzer	Receives data and generates coverage insights

User Management

User

Represents any person who has installed the application. Every user is automatically a data contributor, no separate tester role exists. The consentGiven flag must be true before any data collection begins.

Attributes	Methods
userId, name, email, deviceId, isActive, consentGiven	login(), giveConsent(), viewResults(), triggerSession(), backgroundCollect()

Privacy & Security

Consent Manager

Handles the full lifecycle of user consent — displaying the consent screen, recording agreement with a timestamp, revoking consent, and triggering data deletion. No data collection can occur while isConsented is false. Exists as a separate class to satisfy GDPR auditability requirements.

Attributes	Methods
consentTimestamp, isConsented, userId	showConsentUI(), recordConsent(), revokeConsent(), requestDeletion()

PrivacyAnonymizer

Ensures that no personally identifiable information is stored or transmitted. Replaces real device identifiers with an irreversible HMAC-SHA256 hash combined with a server-side salt. Every sample passes through this class before being saved.

Attributes	Methods
serverSalt, algorithm	hashDeviceId(serial), anonymizeSample(sample)

Core Data

Session

Represents one complete period of data collection. Provides temporal and geographic context for all samples collected while the user is moving. Created automatically on motion detection and closed when the user becomes stationary.

Attributes	Methods
sessionId, startTime, endTime, area, deviceModel, osVersion, triggeredBy	autoStart(), autoStop(), getSamples()

Sample

The core data entity of the system. Each sample is a single network measurement taken at one specific moment. It acts as a container that aggregates RFMetrics, GPSData, and DeviceMetadata into one complete, geo-referenced measurement record.

Attributes	Methods
sampleId, sessionId, deviceHash, timestamp, networkType, appVersion	getRFMetrics(), getGPSData(), getDeviceMeta()

RFMetrics

Stores all radio frequency signal measurements: signal strength (RSRP), quality (RSRQ), clarity (SINR), overall strength (RSSI), and the cell tower identifiers. These values are the primary inputs for coverage analysis.

Attributes	Methods
rsrp, rsrq, sinr, rssi, cellId, pci	validate(), toJson()

GPSData

Records the device's geographic location at the moment of measurement. Includes accuracy filtering (readings worse than 100 m are rejected) and a speed value that drives adaptive sampling — faster movement triggers more frequent samples.

Attributes	Methods
latitude, longitude, altitude, speed, accuracy, timestamp	isValid(), toJson()

DeviceMetadata

Captures hardware and software context about the device taking the measurement. Because antenna hardware and modem firmware vary between models, this context is essential for correctly interpreting signal values.

Attributes	Methods
deviceModel, osVersion, networkType, carrier, appVersion	toJson()

ThroughputMeasurer

Actively measures download and upload speed by running a controlled HTTP speed test. Unlike the other measurement classes which passively read values, this class initiates network activity. It runs at a configured frequency to limit battery and data usage, and attaches its result to a specific sample.

Attributes	Methods
payloadSize, wifiOnly, endpoint	measureDownload(), measureUpload(), attachToSample(sample)

Storage & Synchronization

LocalDatabase

Provides offline-first persistent storage using an on-device SQLite database. Every anonymized sample is written locally immediately after collection, regardless of internet availability. Supports encryption at rest and automatic retention-based cleanup (default 30 days).

Attributes	Methods
dbPath, isEncrypted, retentionDays	saveSample(), getSamples(), deleteSample(), purgeAll()

BatchProcessor

Groups individual samples into batches before upload. A batch is finalized when it reaches 500 samples, 5 MB, has accumulated for over one hour, or when the session ends. Batching drastically reduces HTTP requests, server load, and battery consumption compared to uploading samples individually.

Attributes	Methods
batchId, maxSize, maxBytes, status	createBatch(), addSample(), isFull(), serialize()

SyncEngine

Manages upload scheduling, network condition checking, failure handling, and retry logic. Uses Android WorkManager to guarantee delivery even when the app process is killed. Supports Wi-Fi-only mode and exponential backoff retries. Marks batches as uploaded and instructs LocalDatabase to delete confirmed data.

Attributes	Methods
retryCount, wifiOnly, lastSyncTime	sync(), scheduleUpload(), retryFailed(), markUploaded(batchId)

Backend & Analysis

BackendServer

Represents the remote server the mobile app communicates with over HTTPS. Exposes REST API endpoints for ingesting sample batches, handling deletion requests, delivering remote configuration, and confirming server health before upload attempts.

Attributes	Methods
baseUrl, authToken, apiVersion	ingestSamples(batch), requestDeletion(userId), getConfig(), healthCheck()

CoverageAnalyzer

The final stage of the data pipeline. Processes all collected samples to produce actionable coverage insights: structured quality reports, geographic heatmaps, coverage hole detection, and exportable reports for network operators.

Attributes	Methods
threshold, resolution	analyzeCoverage(), generateHeatmap(), detectCoverageHoles(), exportReport()

4.6.4 Class Relationships

The diagram uses three types of relationship between classes.

4.6.5 Association

Association (solid arrow) means one class holds a reference to another. It represents a persistent has-a link.

- User → Session: A single user can trigger many sessions over time (one-to-many).
- Session → Sample: A single session produces many measurement samples (one-to-many).
- BackendServer → CoverageAnalyzer: The server feeds collected data to the analyzer.

4.6.6 Composition

Composition (filled diamond arrow) is a strong ownership relationship. The child object cannot exist independently of the parent. If the parent is deleted, all composed children are deleted with it.

- Sample → RFMetrics: Each sample owns exactly one set of RF signal measurements.
- Sample → GPSData: Each sample owns exactly one geographic location record.
- Sample → DeviceMetadata: Each sample owns exactly one device context record.

4.6.7 Dependency

Dependency (dashed arrow) means one class uses another to carry out its work. It is a weaker, often temporary relationship — typically a method call.

- ConsentManager uses User to look up and update consent status.
- PrivacyAnonymizer uses ConsentManager to verify consent is active before processing.
- PrivacyAnonymizer uses Sample to replace its identifiable fields with anonymous equivalents.
- ThroughputMeasurer uses Sample to attach measured speed results to a geo-referenced record.
- LocalDatabase uses Sample as the unit it stores and retrieves.
- BatchProcessor uses LocalDatabase to retrieve unuploaded samples.
- SyncEngine uses BatchProcessor to obtain serialised upload payloads.
- SyncEngine uses BackendServer to transmit batches over HTTPS.

4.6.8 Complete Relationship Summary

Relationship	Type	Cardinality	Description
User → Session	Association	1 to many	A user triggers multiple sessions over time
Session → Sample	Association	1 to many	A session contains many measurement samples
Sample → RFMetrics	Composition	1 to 1	Each sample owns one set of RF measurements
Sample → GPSTData	Composition	1 to 1	Each sample owns one GPS location record
Sample → DeviceMetadata	Composition	1 to 1	Each sample owns one device context record
ThroughputMeasurer → Sample	Dependency	attaches to	Speed results are attached to a specific sample
LocalDatabase → Sample	Dependency	stores	The database persists sample objects
BatchProcessor → LocalDatabase	Dependency	reads from	BatchProcessor retrieves samples for grouping
SyncEngine → BatchProcessor	Dependency	uses	SyncEngine requests prepared batches for upload

Relationship	Type	Cardinality	Description
SyncEngine → BackendServer	Dependency	uploads to	SyncEngine transmits batches to the server
BackendServer → CoverageAnalyzer	Association	feeds	The server supplies data for analysis
ConsentManager → User	Dependency	manages consent	ConsentManager controls the user's consent state
PrivacyAnonymizer → ConsentManager	Dependency	uses	Anonymizer verifies consent before processing
PrivacyAnonymizer → Sample	Dependency	anonymizes	Anonymizer replaces identifiers in each sample

4.6.9 End-to-End System Flow

The sequence below traces how data moves through all 15 classes from the moment the user opens the app to the production of a coverage report.

Step	Class(es) Involved	What Happens
1 . Consent	ConsentManager, User	User is shown the consent screen. On agreement, isConsented is set to true and the timestamp is recorded. No data collection can begin without this.
2 . Session Start	User, Session	Motion is detected. triggerSession() creates a new Session; autoStart() begins the collection period.
3 . Sample Collection	Session, Sample, RFMetrics, GPSData, DeviceMetadata, ThroughputMeasurer	backgroundCollect() is called repeatedly. Each call reads signal metrics, GPS coordinates, and device info, and optionally runs a speed test.
4. Anonymisation	PrivacyAnonymizer, Sample	The real device identifier is replaced with an HMAC-SHA256 hash. The sample contains no personally identifiable information after this step.
5. Local Storage	LocalDatabase, Sample	The anonymised sample is immediately written to the on-device SQLite database, regardless of internet availability.

Step	Class(es) Involved	What Happens
6. Session End	Session	The user stops moving. <code>autoStop()</code> closes the session.
7. Batching	BatchProcessor, LocalDatabase	Samples are read from LocalDatabase and grouped into batches of up to 500 samples or 5 MB, then serialised to JSON.
8. Upload	SyncEngine, BackendServer, LocalDatabase	WorkManager triggers <code>sync()</code> . Batches are sent to <code>BackendServer.ingestSamples()</code> . On confirmation, <code>markUploaded()</code> is called and samples are removed from LocalDatabase.
9. Analysis	CoverageAnalyzer, BackendServer	The analyser processes all received samples, generates heatmaps, detects coverage holes, and exports reports for network operators.

4.6.10 Design Principles Reflected in the Diagram

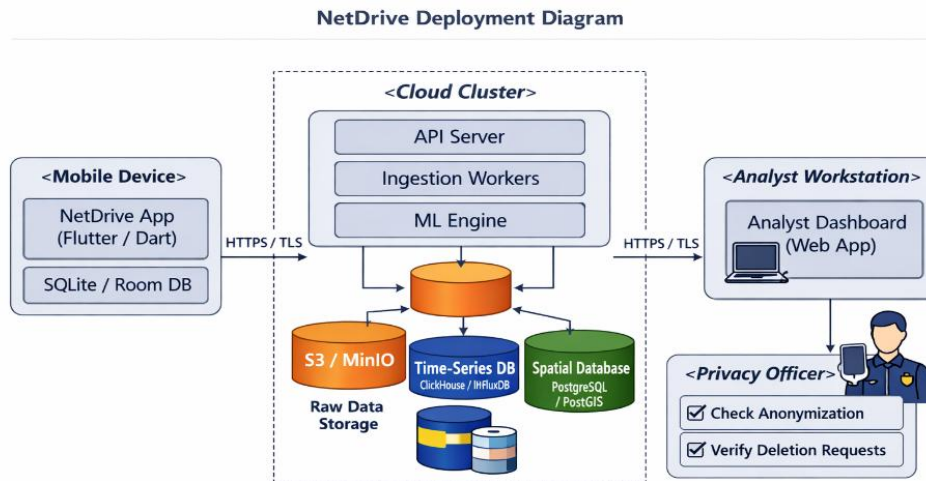
The class structure embodies several key object-oriented design principles:

Principle	How It Appears in the Diagram
Single Responsibility	Every class has one clearly defined job. <code>PrivacyAnonymizer</code> only anonymizes. <code>BatchProcessor</code> only groups samples. No class handles multiple unrelated concerns.
Separation of Concerns	Each of the five functional groups handles a distinct layer. Privacy logic never touches storage logic; storage logic never touches analysis logic.
Encapsulation	Internal details are hidden behind methods. For example, the hashing algorithm inside <code>PrivacyAnonymizer</code> is not visible to any other class — only <code>hashDeviceId()</code> is exposed.
Composition over Inheritance	<code>Sample</code> is composed of <code>RFMetrics</code> , <code>GPSData</code> , and <code>DeviceMetadata</code> rather than using a deep inheritance hierarchy. This makes the data model flexible and easy to extend.
Offline-First Design	Data is always written to LocalDatabase before any upload is attempted, ensuring the app works correctly in areas with no connectivity and data is never lost.

4.7 Deployment Diagram

4.7.1 Purpose

The Deployment Diagram illustrates the physical architecture of the NetDrive system, showing how software components are distributed across hardware nodes and how they interact through secure connections.



4.7.2 Description

- Mobile Device (Android/iOS):
 - Runs the NetDrive App (Flutter/Dart).
 - Stores collected data in SQLite/Room DB.
 - Communicates securely with the cloud cluster.
- Cloud Cluster (Docker + Kubernetes):
 - API Server: Manages data ingestion and client communication.
 - Ingestion Workers: Validate and preprocess incoming data.
 - ML Engine: Performs coverage prediction and hole detection.
 - Databases:
 - PostgreSQL/PostGIS: Spatial queries and map generation.
 - ClickHouse/InfluxDB: Time-series analytics.
 - S3/MinIO: Raw data storage and backup.
- Analyst Workstation:
 - Hosts the Analyst Dashboard (Web App) for visualization and reporting.
 - Connects securely to the cloud cluster via HTTPS/TLS.
- Privacy Officer Node:
 - Oversees anonymization and deletion compliance.
- maintain transparency and compliance.

4.7.3 Conclusion

The Data Flow and Deployment Diagrams together provide a comprehensive view of the NetDrive system's architecture:

- The DFD captures how data moves logically through the system.
- The Deployment Diagram shows how components are physically distributed and interact securely. Together, they demonstrate a robust, scalable, and privacy-conscious design suitable for real-world network measurement and analysis.
- The deployment architecture ensures data security, scalability, and modular service orchestration.
- Cloud-based components are containerized for efficient resource management.
- Analyst and privacy layers maintain transparency and compliance.

References

1. Sommerville, I. (2016). *Software Engineering* (10th Edition). Pearson Education. – Provides foundational concepts for system modeling, UML diagrams, and architectural design.
2. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th Edition). McGraw-Hill. – Covers data flow diagrams, deployment diagrams, and their role in system documentation.
3. Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd Edition). Addison-Wesley. – Reference for UML notation including composition, aggregation, associations, and dependencies.
4. National Science Foundation (NSF). (2020). *Crowdsourced Mobile Network Measurement Systems* [Research Publication]. Available at: [NSF PAR Repository](#) – Discusses architectures for crowdsensed mobile network measurement, relevant to NetDrive's design.
5. Object Management Group (OMG). (2017). *Unified Modeling Language (UML) Specification, Version 2.5.1*. – Official UML specification defining diagram types, symbols, and cardinalities.
6. Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd Edition). Addison-Wesley. – Provides architectural principles for deployment diagrams and layered system design.